07 — Error Level Analysis & copy-move detection

What they detect: *pixel-level* evidence that part of a page was edited after the rest was last saved (ELA), or that a region was copied from one place on a page to another (copy-move). Together they look for pasted-over totals, cloned signatures/stamps and patched line items.

Code: `services/forensics/ela.py` (`run_ela_check`), `services/forensics/copy_move.py` (`run_copy_move_check`), `services/forensics/pdf_render.py` (`render_pdf_pages`), run by `tasks/tampering_checks_task.py` (`run_tampering_checks`). Two `document_checks` rows result: `error_level_analysis` and `copy_move_detection` .

Shared render

`render_pdf_pages(pdf_bytes) -> list[RenderedPage]` rasterizes every page **once** at **200 DPI** with PyMuPDF into BGR `numpy` arrays and hands the same list to both checks (the render is the expensive part, so `run_tampering_checks` does it once). The duplicate-detection, visual-review and signature tasks render the PDF separately with the same function.

```
@dataclass RenderedPage:
    page_number: int          # 1-based
    image: np.ndarray         # BGR uint8
    width: int; height: int
    has_image_content: bool   # page has ≥1 embedded raster image XObject
```

200 DPI is chosen to resemble a real scan: rendering far higher would exaggerate ELA's block artifacts and make its known print-and-rescan weakness worse.

ELA (Error Level Analysis)

Idea: JPEG-recompress the page at a fixed quality and diff it against the original. A region that was edited after the last save recompresses differently from the rest of the page.

Algorithm (adapted from an internal reference tool): recompress at JPEG quality **75**; take the non-linear `sqrt(diff)` error with a gain; threshold at the **99.7th percentile** with an absolute floor of 60; group into regions; keep regions between **0.2 %** and **60 %** of the page area. `_detect_anti_forensics` separately flags a page that is *suspiciously clean*: a median-blur noise variance below 5.0 ("possible anti-forensic smoothing") or a Laplacian standard deviation below 1.5 ("possible over-compression") — the signature of someone smoothing or re-compressing a page to defeat ELA.

Gate: only pages with `has_image_content` are analyzed. No such page → `not_applicable` .

Findings: `recompression_error_region` (medium) — with `page` , `bounding_box` and `data.region_area_fraction` ; `anti_forensic_signal` (high).

Copy-move detection

Idea: find a region duplicated elsewhere on the *same page* — a common way to hide or fabricate content.

Algorithm (adapted from an internal reference tool, pipeline unchanged apart from one fixed bug):

1. **BRISK** keypoints on the grayscale page (detector threshold 200), keeping those whose normalized response is above `100 – 90` .

2. **Radius matching** of each keypoint's descriptor against all others (Hamming distance ≤ 20 % of 255 ≈ 51), excluding self-matches.
3. **Displacement filter:** discard matches whose two points are closer than `0.15 × min(page height, width) / 2` — nearby points are not "moved" content. That same distance bounds the grouping in the next step.
4. **Cluster** matches that share a similar displacement and whose endpoints sit near each other; keep clusters of **≥ 30** matches.
5. **De-duplicate** overlapping clusters (IoU > 0.3) and keep regions covering ≥ **2 %** of the page.

It runs on **every** page, image-bearing or not: a duplicated paragraph or logo shows up as a keypoint cluster whether the source is a photo or vector text.

Safety valves: if a page yields more than 4000 keypoints after filtering, or more than 4000 matches after the displacement filter, the page is skipped and returns **no clusters** (the clustering loop is $O(n^2)$). A very dense page therefore *cannot* produce a copy-move finding.

Finding: `copy_move_cluster` (high) with `page`, `bounding_box` and `data.matched_pairs`. No visualization image is generated; the frontend draws boxes live.

Output shape (real)

Both checks return `{"result": "pass"|"flag", "details": [Finding, …]}`; ELA can also return `"not_applicable"`. When a page's text is **vector text drawn over an image** (a scan converted to editable text — `services/forensics/page_structure.py`), a clean result is reported as **`"limited"`** instead of `"pass"`, with an info finding `pixel_analysis_limited`: pixel checks cannot see an edit to that text, so "pass" would give false comfort. A flag stays a flag.

```
// error_level_analysis
{"result": "flag", "details": [{
  "finding": "recompression_error_region", "severity": "medium", "page": 1,
  "description": "Page 1: a localized area shows a higher JPEG recompression error than the rest of the
page",
  "bounding_box": {"page": 1, "x": 0.5526, "y": 0.4951, "width": 0.3688, "height": 0.0338},
  "data": {"region_area_fraction": 0.0094}}]}

// copy_move_detection
{"result": "flag", "details": [{
  "finding": "copy_move_cluster", "severity": "high", "page": 1,
  "description": "Page 1: found a cluster of 58 matching feature pairs, consistent with one region of this
p…",
  "bounding_box": {"page": 1, "x": 0.348, "y": 0.3164, "width": 0.1469, "height": 0.4271},
  "data": {"matched_pairs": 58}}]}
```

The `Finding` dataclass is `{finding, severity, description, page?, bounding_box?, data?}`.

In the UI

Boxes are drawn live over the PDF by `PdfOverlayViewer` — **ELA solid red, copy-move solid orange** — and in the report's Section 9. Nothing annotated is ever stored by the live UI.

Known limitations

- **ELA is a signal, not a verdict.** It is weak against print-and-rescan tampering and prone to false positives on legitimately recompressed scans. This is why the seeded weights are moderate.

- **Copy-move flags repeated legitimate structure** (table rows, repeated logos or headers, identical stamps) — it has no notion of "expected" repetition.
- Both are PDF-only and both render at a fixed DPI; very small edits can fall below the region-area floors.
- Copy-move silently produces nothing on very dense pages (the 4000-keypoint / 4000-match caps above).
- Findings on documents with no raster content will never come from ELA (`not_applicable`); a pure-text forged PDF is caught, if at all, by metadata, copy-move, visual review and field checks.

On such a page the background itself is searched for traces of text deleted or shortened after the conversion — the ghost-content check, 07a, run by the same task.

Risk rules fed

`rule_id`	Check / finding	Weight	Severity
`ela.tamper_region_detected`	ELA / `recompression_error_region`	25	medium
`ela.anti_forensic_signal`	ELA / `anti_forensic_signal`	30	high
`copy_move.cluster_detected`	copy-move / `copy_move_cluster`	35	high